```
    def subject
      @subject ||= connection.get(path_prefix){|req| req.params.merge!params }.body
    end
  end
end
```

Before you understand this class, you need to see how Ghee uses it. For each type of GitHub object, such as *gists* or *users*, Ghee defines one subclass of Ghee::ResourceProxy. Here is the class for gists (the class for users is quite similar):

gems/ghee-0.9.8/lib/ghee/api/gists.rb

```
class Ghee
  module API
    module Gists
      class Proxy < ::Ghee::ResourceProxy
        def star
          connection.put("#{path_prefix}/star").status == 204
        end

        # ...

    end
  end
end
```

When you call a method that changes the state of an object, such as Ghee::API::Gists#star, Ghee places an HTTP call to the corresponding GitHub URL. However, when you call a method that just reads from an attribute, such as url or description, that call ends into Ghee::ResourceProxy#method_missing. In turn, method_missing forwards the call to the object returned by Ghee::ResourceProxy#subject. What kind of object is that?

If you dig into the implementation of ResourceProxy#subject, you'll find that this method also makes an HTTP call to the GitHub API. The specific call depends on which subclass of Ghee::ResourceProxy we're using. For example, Ghee::API::Gists::Proxy calls https://api.github.com/users/nusco/gists. ResourceProxy#subject receives the GitHub object in JSON format—in our example, all the gists of user *nusco*—and converts it to a hash-like object.

Dig a little deeper, and you'll find that this hash-like object is actually a Hashie::Mash, the magic hash class that we talked about in *The Hashie Example*, on page 57. This means that a method call such as my_gist.url is forwarded to Ghee::ResourceProxy#method_missing, and from there to Hashie::Mash#method_missing, which finally returns the value of the url attribute. Yes, that's two calls to method_missing in a row.